Introduction to the Yale SOM HPC

A guide for researchers whose code has, until now, lived on a laptop

Contents

What is an HPC, and why use one? . . . 2
The mental model: a shared instrument . . . 2
Login nodes vs compute nodes . . . 3
Connecting to the cluster . . . 3
The job scheduler: thinking in jobs, not processes . . . 4
Right-sizing: the post-job feedback loop . . . 6
The filesystem zoo . . . 6
Moving files in and out of the HPC . . . 8
Software: modules, environments, and no sudo . . . 9
Running Python . . . 10
Running R . . . 11
Running Stata . . . 12
GPUs . . . 13
Working with large data . . . 14
Acquiring data: WRDS, APIs, scraping . . . 16
Starting a new project . . . 17
HPC etiquette . . . 18
Working with Claude Code on the HPC . . . 19
Getting help . . . 19
Appendix: example files . . . 21

> Claude Code skills: `overview`, `managing-jobs`
> This guide has a companion: the Yale SOM Claude Code marketplace, a plugin that ships fifteen skills — AI-facing counterparts to the sections below, covering `overview`, `managing-jobs`, `using-gpus`, `running-python`, `running-r`, `acquiring-data`, and the rest. When you ask Claude Code to do something on the cluster, it loads the matching skill and follows the same conventions this document teaches you. Each section ends with a callout naming the relevant skill or skills; Section 18 has the full map and install instructions.

What is an HPC, and why use one?

On a laptop, you have a fixed budget of cores, memory, and time, and you control all three. Once the job exceeds any one of them, you wait. The Yale SOM HPC exists to give you a much larger budget — and to give it to you on terms set by a shared scheduler rather than by your hardware.

An HPC ("high-performance computing") cluster is a collection of large servers — *compute nodes* — sitting in a data center, sharing a fast filesystem, and managed by a *job scheduler* that hands out CPU cores, memory, GPUs, and time to many users at once. The Yale SOM cluster lives at `hpc.som.yale.edu`. You log in to a small *login node*, and all real work runs on the compute nodes after the scheduler grants you a slot.

The cluster is the right tool when at least one of five things is true: your data does not fit in your laptop's RAM; a run takes long enough that you do not want to babysit your laptop through it; you want to run the same script hundreds of times with different parameters in parallel; you need a GPU you do not own; or the work is sensitive enough that it should live on Yale-managed storage rather than a personal device. The cluster is *not* the right tool for a 30-second script, for interactive plotting, or for work that fits comfortably on the machine you already have. The right move is to migrate to the cluster when the laptop stops being enough, not out of habit.

This guide does two things. First, it explains the mental model — what is different about a shared, scheduled, multi-tenant compute environment, and why almost every operational rule on the cluster follows from that one fact. Second, it provides the practical recipes — Slurm scripts, module loads, file-transfer patterns, language-specific workflows — you will use every day.

We structure the rest of the guide as follows. Sections 2–4 cover the mental model, the login-versus-compute distinction, and how to connect. Sections 5–7 cover Slurm, the post-job feedback loop, and the filesystem. Sections 8–9 cover moving files in and out and installing software. Sections 10–12 cover Python, R, and Stata. Sections 13–15 cover GPUs, larger-than-memory data, and acquiring data from outside the cluster. Sections 16–17 cover project setup and etiquette. Section 18 explains how this guide pairs with the Claude Code marketplace, which encodes the same conventions for the AI you delegate to. Section 19 lists the commands and contacts to keep at hand, and the appendix gives a complete first-job example.

> Claude Code skill: `overview`
> The `overview` skill is the AI's version of this section. When you ask Claude Code to do anything on the cluster without specifying further, it loads `overview` first to get its bearings.

The mental model: a shared instrument

The single biggest adjustment from working locally is that the HPC is shared, right now, with everyone else in SOM. Your laptop is yours. Latency is zero, nothing is queued, and you never wait for jobs to run. The HPC is the opposite of all of those, and operational rules on the cluster follow from this difference.

To see what that means in practice, four facts are worth keeping in mind together. CPUs, memory, and GPUs are a finite pool divided across many users, so an inflated request makes

you wait longer in the queue *and* blocks someone else. Resources are also reserved up front rather than discovered at runtime — you declare what you need before a job starts, and Slurm enforces the declaration by killing any job that exceeds it. Usage is public; `squeue` shows what every user on the cluster is running, so a 64-CPU job left idle overnight is visible by name to the people waiting behind it. And side effects propagate: millions of tiny files slow the GPFS metadata server for everyone, an aggressive scraper gets the cluster's outbound IP blocked for everyone, and a misbehaving login-node process lags everyone else's editor.

The Yale SOM HPC marketplace summarizes the implications as two pillars. *Be polite*: do not hog, do not sit idle, do not trash GPFS metadata, do not get the cluster IP-blocked. *Be skillful*: right-size requests, control threads, cache expensive work, make outputs resumable, and inspect resource use after every serious job. Everything that follows in this guide is an elaboration on one or the other.

Login nodes vs compute nodes

When you `ssh` to the cluster, you land on a *login node*. The login nodes are small machines shared by every logged-in user, and they exist for editing files, running `git`, browsing modules with `module`
`spider`, doing small package installs, and submitting jobs. They are not for running an analysis, training a model, loading a 5 GB CSV into pandas, or anything else that pegs a CPU or holds significant RAM. When someone runs heavy compute on a login node, every other user's editor lags and the sysadmins notice.

The right way to "just try something quickly" is an interactive job on a compute node:

```
srun --partition=cpunormal --cpus-per-task=2 --mem=8G --time=01:00:00 --pty bash
```

This allocates two cores, 8 GB of memory, and one hour of walltime, and drops you into an interactive shell on a real compute node. Exit the moment you are done — `Ctrl+D` or `exit` — rather than letting it sit open while you go to a meeting.

> Claude Code skills: `overview`, `managing-jobs`
> Claude Code knows not to run heavy commands on the login node, and reaches for `srun --pty` or `sbatch` instead. If it ever proposes running a long-running command on a login node, push back.

Connecting to the cluster

The basic form of connecting is the same as any SSH session:

```
ssh <your-netid>@hpc.som.yale.edu
```

That works, but for daily use it is worth three small upgrades. First, generate an SSH key with a passphrase on your laptop using `ssh-keygen -t ed25519`, and copy only the public half to the cluster with `ssh-copy-id <netid>@hpc.som.yale.edu`. Never copy your private key (`id_ed25519`) onto GPFS. Second, run an SSH agent — `ssh-agent` on Linux, the macOS keychain on a Mac — so that you type the passphrase once per session. Third, add a host block to `~/.ssh/config` so that the full hostname is no longer required:

```
Host somhpc
  HostName hpc.som.yale.edu
  User <your-netid>
```

Enable `ForwardAgent yes` only when you need it for GitHub or another SSH hop. Agent forwarding is convenient, but any process on the remote host can ask your agent to sign while that connection is active. For GitHub from the cluster, prefer `gh auth login` when the GitHub CLI is available; carefully scoped agent forwarding is the alternative. If GitHub auth worked yesterday and fails today inside `tmux`, do not copy keys to the cluster. Start a fresh SSH login or refresh `SSH_AUTH_SOCK`; the socket path went stale, not the private key.

You must be on a Yale network path — campus network or Yale VPN — to reach the cluster. Fix network access before debugging keys. Only SSH to a compute node after Slurm has allocated that node to you; use the hostname printed inside the allocation, then tunnel to that node for Jupyter or VS Code. Do not run notebook kernels or heavy editor extensions on the login node.

> Claude Code skill: `connecting-securely`
> Full recipes for SSH keys, agent forwarding, stale agent sockets, Jupyter/VS Code tunnels, and compute-node SSH live in `connecting-securely`. Ask Claude Code to "set up an SSH config for the SOM HPC" and it will follow that skill.

The job scheduler: thinking in jobs, not processes

On your laptop you run a process — `python script.py` — and it starts immediately. On the HPC you submit a *job*: a description of what resources you need and what to run with them. Slurm queues the job, finds a compute node with a slot that fits, and only then starts your code. If your job tries to use more memory or time than you declared, Slurm kills it. The contract is simple: you predict what you need; Slurm enforces it.

The contract has four axes — CPU cores, memory, walltime, and GPUs — declared with the directives in Table 1.

Axis	*Slurm directive*
CPU cores	`--cpus-per-task=N`
Memory	`--mem=16G` (per node) or `--mem-per-cpu=4G`
Walltime	`--time=HH:MM:SS`
GPUs	`--gres=gpu:1` or `--gpus=1` (only when needed)

Table 1: The four resource axes of a Slurm job.

A fifth knob, `--ntasks`, is the one most likely to confuse a new user. The plain-English distinction is that `--ntasks=N` requests N independent processes, typically MPI ranks, while `--cpus-per-task=N` requests N cores assigned to a single process so it can run threads. SOM workflows are almost never MPI, so almost every sbatch script should start with `--ntasks=1 --cpus-per-task=N`. Reach for `--ntasks>1` only when you have multiple cooperating processes.

A *partition* is a queue tied to a particular set of nodes. The SOM cluster currently exposes five. The default, `default_queue`, mixes CPU-only and A40 GPU nodes and caps walltime at four hours, which makes it well-suited to short tests and most interactive work. `cpunormal` is

CPU-only with longer walltimes and is the workhorse for batch CPU jobs. `gpunormal` carries RTX 8000 and A100 GPUs. The `h100` partition is a single node with four H100 GPUs and is the scarcest resource on the cluster — every H100-hour you take is compute someone else cannot use. The `build` partition exists for compiling software. The live view is `sinfo -s`, which is worth running before any production submission.

A minimal sbatch script that puts the contract on paper looks like:

```
#!/bin/bash
#SBATCH --job-name=test
#SBATCH --partition=default_queue
#SBATCH --time=00:10:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=2G
#SBATCH --output=logs/%x_%j.out

set -euo pipefail

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export MKL_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export NUMEXPR_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export PYTHONUNBUFFERED=1

hostname
python --version
```

The thread-environment block matters more than it looks; we return to it in Section 10. Submit with `sbatch my_job.sh`, monitor your queue with `squeue --me`, inspect a particular job with `scontrol show job <id>`, and cancel with `scancel <id>`. Job states you will see in practice are `R` (running), `PD` (pending in the queue), and `CG` (completing).

If `sbatch job.sh` fails with `bad interpreter`, `: not found`, or `$'\r'`, the script has Windows CRLF line endings. Fix it with `dos2unix job.sh`, or with `sed -i 's/\r$//' job.sh` if `dos2unix` is not installed, and configure your editor to write LF for `.sh` files.

Interactive sessions are for debugging, not unattended work. Use the smallest allocation that lets you debug, put a short time limit on it, and exit the moment you are done:

```
srun --partition=cpunormal --cpus-per-task=2 --mem=8G --time=01:00:00 --pty bash
```

If a job sits in `PD` for an unusually long time, ask why. The right diagnostic is `squeue --me --start`, which gives the estimated start time and the queueing reason, followed by `scontrol show job <id>` for the full record. The most common cause of pending forever is requesting resources that do not exist on any node: 256 GB of RAM when the largest node has 192 GB, or 64 cores on partitions whose nodes top out at 48. Slurm queues such a job indefinitely without warning that it is impossible. Match the request to real hardware by inspecting `sinfo -s` and `scontrol show node`.

After a job finishes, `sacct -j <id>` reports its history with whatever columns you ask for:

```
sacct -j <id> --format=JobID,JobName,State,Elapsed,MaxRSS,ReqMem,AllocCPUS
```

Two patterns worth knowing about up front cover most of the structured work on a cluster. *Job arrays* answer "I want to run this script 500 times with different parameters" — add `#SBATCH --array=1-500%50` and the `%50` caps concurrent tasks at 50, leaving slots for other users; inside the script, `$SLURM_ARRAY_TASK_ID` is the index. *Dependencies* answer "B should start when A finishes": `sbatch --dependency=afterok:<jobid-of-A> B.sh`.

> Claude Code skills: `managing-jobs`, `self-diagnosing-resource-use`
> The `managing-jobs` skill has the full pattern library — array throttling, dependency chains, partition selection, the right-sizing loop. Pair it with `self-diagnosing-resource-use` for post-mortems.

Right-sizing: the post-job feedback loop

Resource requests are guesses. The discipline that distinguishes a skillful HPC user from a wasteful one is checking the guess after the job finishes and adjusting next time. Two commands do most of the work:

```
seff <job-id>
sacct -j <job-id> --format=JobID,Elapsed,MaxRSS,ReqMem,AllocCPUS,State
```

What does "good" look like on the resulting numbers? CPU efficiency above about 50% is reasonable for CPU-bound work; 10-50% often means I/O-bound work or an over-request; below 10% is probably wasteful. If CPU efficiency is low, request fewer CPUs or fix the parallelism before scaling up. For memory, set the next `--mem` to roughly 1.5-2× the observed peak `MaxRSS`, not 10×. If you asked for 128 GB and used 4 GB, you removed 124 GB from everyone else's available pool for no reason. Walltime should come from a sample-data extrapolation; if work is resumable, prefer 1-4 hour chunks that can backfill into idle slots rather than one multi-day job.

Treat future caps as if they already exist. SOM HPC is still lightly enforced compared with many clusters, but hard per-user CPU, memory, GPU, or interactive limits are the natural direction for a shared instrument. Right-sizing now keeps your work schedulable later.

Why bother? Over-requesting is not free. It costs *you* a longer queue because Slurm has to find a bigger empty slot for your job, and it costs other users the resources sitting idle inside your allocation. Inspect every serious job; adjust the next submission. The habit is worth more in the aggregate than any single performance optimization.

> Claude Code skill: `self-diagnosing-resource-use`
> Ask Claude Code "did my last job use what it asked for?" and it will run `sacct` and `seff`, interpret the output, and propose a tighter request for next time.

The filesystem zoo

Your laptop has one disk. The cluster has several filesystems, with different rules and lifetimes, summarized in Table 2. Knowing which is which is the difference between a workflow that runs cleanly and one that quietly fills `$HOME`, breaks shared metadata, or loses intermediates when a node reboots.

Path	*What it is for*	*Lifetime*
`$HOME` (= `/home/$USER`, same as `/gpfs/home/$USER`)	Code, configs, dotfiles, small things. Backed up.	Permanent
`/gpfs/project/<proj>/`	Shared team data, code, outputs. Request via `somit@yale.edu`.	Project lifetime
`/gpfs/scratch60/$USER/`	Large temporary working files; staging area.	Treat as ephemeral; clean it
`/tmp` on a compute node	Local fast scratch, just for this job.	Gone when the job ends

Table 2: The four filesystems you will use, and what each is for.

The implications are short. Code lives in `$HOME` or in `code/` under your project; data lives under `/gpfs/project/<proj>/data/` or `/gpfs/scratch60/$USER/`. `$HOME` is small and is the wrong place for 200 GB of intermediate files. `/gpfs/scratch60/$USER` may not exist until you create it with `mkdir -p /gpfs/scratch60/$USER`. Scratch is a staging area, not an archive — clean it when work is done.

The metadata-storm warning

GPFS is good at large files and bad at millions of tiny ones. Every file creation, listing, or stat call hits a metadata server shared by every user on the cluster. A workflow that writes one output file per row of a 2-million-row dataset will slow `ls` for every user, including the sysadmins, until you stop. Three counter-patterns cover most of the situations where this comes up: write Parquet rather than per-row CSVs; tar or zip directories of small files when you are done with them; and for "many small intermediates inside a single job," write to the compute node's `/tmp` and copy a single tarball back to GPFS at the end.

Atomic writes and resumable outputs

Jobs can be killed. You exceed memory, the wallclock runs out, or the node reboots for maintenance. If your script writes results directly to `output.parquet` and is killed halfway, you now have a half-written `output.parquet` that looks like a finished file but is not. The safe pattern is to write to a temporary path on the same filesystem and then rename:

```
import os
tmp = "output.parquet.tmp"
df.write_parquet(tmp)
os.replace(tmp, "output.parquet")  # atomic on the same filesystem
```

Combine this with skip-if-exists at the top of each unit of work and the pipeline becomes resumable: re-running the same script after a kill or a crash picks up where it left off rather than restarting from zero. For arrays, the usual shape is one output file per task — for example `output/task_0001.parquet`, `output/task_0002.parquet` — then a final combine step. That is far better than thousands of per-row CSVs and safer than many tasks mutating one large file in place.

For high-I/O work inside one job, stage onto compute-node `/tmp` and copy only final outputs back to GPFS:

```
workdir=$(mktemp -d "${TMPDIR:-/tmp}/job_${SLURM_JOB_ID:-local}.XXXXXX")
export TMPDIR="$workdir"
trap 'rm -rf "$workdir"' EXIT

cp /gpfs/project/<proj>/data/input.parquet "$workdir"/
srun .venv/bin/python src/process.py \
  --input "$workdir/input.parquet" \
  --output "$workdir/output.parquet" &
job_pid=$!
wait "$job_pid"
cp "$workdir/output.parquet" /gpfs/project/<proj>/output/
```

The `cmd & wait $!` shape lets bash run the cleanup trap when Slurm sends `SIGTERM` near the time limit. `SIGKILL` still bypasses traps, which is why `/tmp` is only scratch.

> Claude Code skills: `using-the-filesystem`, `working-with-large-data`
> `using-the-filesystem` has the full pattern library; `working-with-large-data` covers the Parquet / DuckDB / Polars patterns that avoid both metadata storms and out-of-memory kills.

Moving files in and out of the HPC

A new user's first concrete obstacle is rarely Slurm. It is getting an existing codebase and dataset onto the cluster in the first place, and getting results back off. Three situations cover most of what comes up.

I already have a codebase locally

Use Git, not `scp`. Push your repo to GitHub or to Yale's GitHub Enterprise, and on the cluster:

```
cd /gpfs/project/<proj>/
git clone git@github.com:<you>/<repo>.git code
```

From then on, you edit anywhere — laptop, cluster shell, VS Code Remote-SSH — and `git pull` and `git push` keep the two in sync. Two free wins follow that are worth stating explicitly.

The first is an off-cluster backup of your code, with no admin involved: the SOM HPC does not give you user-controlled backups of `$HOME` or `/gpfs/project/`, but if your code is on GitHub then a cluster outage is irrelevant to your code's safety. The second is version history for the inevitable "what did I change last week." Commit before each serious run, print the commit hash into your job log, and `git bisect` when a result stops reproducing.

I have a dataset on my laptop

For one-shot transfers, `scp` is fine:

```
scp local.csv somhpc:/gpfs/project/<proj>/data/
```

For anything large, slow, or interruption-prone — which means most real datasets — prefer `rsync`, which resumes:

```
rsync -avP --partial local_dir/ somhpc:/gpfs/project/<proj>/data/
```

Do not check data into Git. That is what `/gpfs/project/` is for. (Git LFS exists; it is rarely worth the trouble for SOM workflows.)

I have results on the cluster I want back

Same tools, reversed direction:

```
rsync -avP --partial somhpc:/gpfs/project/<proj>/output/ ./output/
```

VS Code Remote-SSH makes single-file fetches transparent — opening the file in the remote editor downloads it as needed. The pattern worth avoiding is pulling thousands of small output files individually; tar them on the cluster first, both to spare GPFS metadata (Section 7) and to make the transfer one fast operation rather than thousands of slow ones:

```
tar -czf results.tar.gz output/
```

then `scp` the one tarball.

I want to fetch from a public URL

`wget` or `curl` straight onto `/gpfs/project/`, ideally from a compute node so the login node is not tied up. The cluster's outbound IP is shared, so credentialed downloads, APIs, and rate-limiting deserve the more careful treatment we give them in Section 15.

What does not belong on the HPC

Final-output PDFs, slide decks, Overleaf projects, and manuscript drafts belong on your laptop, in Overleaf, or in Dropbox. The cluster is for compute and the data feeding it. Build the figure on the cluster, copy it to your laptop, drop it into the paper.

> Claude Code skills: `connecting-securely`, `using-the-filesystem`, `starting-a-new-project`
> Ask Claude Code to "clone my GitHub repo into project space and set up a uv environment" and it will combine these three skills.

Software: modules, environments, and no sudo

The biggest software difference from your laptop is that you do not have `sudo`. You cannot `apt install`, `brew install`, or `pip install --system` your way out of a problem. Software on the cluster comes in three layers, in roughly this order of preference: cluster-managed modules, user-space environments, and containers.

The first layer is *modules*, served by Lmod and built behind the scenes with Spack. Find software with `module spider <name>`, load it with `module load <name>`, view the current environment with `module list`, and clear inherited state with `module purge`. Git, R, Stata, CUDA, MATLAB, Apptainer, and sometimes Python are module-provided; load required modules explicitly in scripts rather than relying on your interactive shell.

The second layer is *project-local environments*. For Python, the current default is `uv`: it is fast on GPFS, creates a `.venv/`, and commits a lockfile (`uv.lock`) that reproduces the environment. Avoid `pip install --user`, avoid package installs inside jobs or job arrays, and do not share one `$HOME`-level environment across unrelated projects. For R, use `renv` per project and restore once during setup, not inside hundreds of jobs.

The third layer is *containers*, via Apptainer, for cases where modules and user-space environments are not enough — a GLIBC-too-old error, complex C/CUDA dependencies, or a published Docker image you need to reproduce. Load it with `module load apptainer`. Prefer static or musl Linux binaries for small user tools under `~/.local/bin` when available; they avoid many `GLIBC_2.xx not found` failures.

> Claude Code skills: `installing-software`, `running-python`, `running-r`
> `installing-software` is the umbrella skill; the language-specific ones layer on top. Ask Claude Code to "install package X" and it will pick the right layer (module, uv, renv, Apptainer) for you.

Running Python

For new Python projects, use `uv` and keep the environment under the project code directory:

```
cd /gpfs/project/<proj>/code
uv init --app
uv add polars pyarrow duckdb
uv sync --frozen
```

Commit `pyproject.toml` and `uv.lock`. Do not commit `.venv/`. Run `uv sync --frozen` at setup time on the login node, not inside Slurm jobs or arrays; mutating environments in flight is slow, non-reproducible, and rough on GPFS metadata. Plain `pip` is not forbidden, but `pip install --user` and per-job installs are the bad patterns.

A typical Python sbatch script looks like the following. Note the thread-environment block and the `srun .venv/bin/python` launch line:

```
#!/bin/bash
#SBATCH --job-name=py_analysis
#SBATCH --partition=default_queue
#SBATCH --time=01:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --output=logs/%x_%j.out

set -euo pipefail

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export MKL_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
```

```bash
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export NUMEXPR_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export PYTHONUNBUFFERED=1

cd /gpfs/project/<proj>/code
srun .venv/bin/python src/main.py
```

Use `srun .venv/bin/python ...` for long or resumable jobs so Slurm signals reach the Python process. For short exploratory jobs where signal-based shutdown does not matter, `uv run python src/main.py` is acceptable. In Python itself, read Slurm values with fallbacks so the same code works locally and on the cluster:

```python
import os
n_cpus = int(os.environ.get("SLURM_CPUS_PER_TASK", "0")) or os.cpu_count() or 1
job_id = os.environ.get("SLURM_JOB_ID", "local")
```

Why the thread-environment block matters

NumPy, SciPy, scikit-learn, Polars, NumExpr, and much of the scientific Python stack call into threaded native libraries. By default, those libraries may spawn one thread per CPU they see on the *node*, not per CPU you *requested* from Slurm. The `*_NUM_THREADS` exports pin them to exactly the cores Slurm gave you. The `${SLURM_CPUS_PER_TASK:-1}` form matters: a bare `$SLURM_CPUS_PER_TASK` is empty outside Slurm.

Python data defaults and resumable outputs

For tabular work, prefer Polars lazy scans and DuckDB before reaching for multiprocessing. Store reusable data as Parquet with compression, convert to pandas only at library boundaries, and write one Parquet per array task or chunk rather than mutating one large file in place.

Write scripts so each unit of work skips if its output already exists, writes to `<name>.tmp`, and renames only after success:

```python
from pathlib import Path
import polars as pl

output = Path("/gpfs/project/<proj>/output/task_0001.parquet")
if output.exists():
    raise SystemExit(0)

tmp = output.with_suffix(".parquet.tmp")
pl.DataFrame({"ok": [1]}).write_parquet(tmp)
tmp.rename(output)
```

> Claude Code skills: `running-python`, `parallel-python`, `accelerating-python`
> Three layered skills. `running-python` covers uv, Slurm, logging, and resumable outputs. `accelerating-python` covers DuckDB, Polars, and Numba before parallelism. `parallel-python` covers worker sizing, arrays, and signal handling once you actually need workers.

Running R

R loads through the module system, and job scripts should load it explicitly:

```bash
module spider r
module load r
R --version
```

Project-local package management uses `renv`. Initialize and restore on the login node during setup, commit `renv.lock` and `.Rprofile`, and do not commit `renv/library/`:

```r
install.packages("renv")
renv::init()
renv::install(c("data.table", "arrow", "fixest"))
renv::snapshot()
```

For shared projects, put the renv library under project space by adding to `.Rprofile`:

```r
Sys.setenv(RENV_PATHS_LIBRARY = "/gpfs/project/<proj>/environments/renv/library")
```

Run `renv::restore()` once during setup, not inside a Slurm array. A typical R sbatch script:

```bash
#!/bin/bash
#SBATCH --job-name=r_analysis
#SBATCH --partition=default_queue
#SBATCH --time=01:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --output=logs/%x_%j.out

set -euo pipefail

module purge
module load r

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export MKL_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}

cd /gpfs/project/<proj>/code
srun Rscript src/main.R
```

Inside R, match package threads to the Slurm allocation:

```r
slurm_cpus <- Sys.getenv("SLURM_CPUS_PER_TASK", "")
n_cpus <- if (nzchar(slurm_cpus)) as.integer(slurm_cpus) else parallel::detectCores()
data.table::setDTthreads(n_cpus)
```

For ordinary research code, tidyverse is readable and shareable; switch to `data.table` or `dtplyr` when you have measured memory or runtime as the bottleneck. Use Arrow + Parquet for reusable data on GPFS.

> Claude Code skill: `running-r`
> `running-r` covers `renv` setup, batch invocation, and the BLAS, OpenMP, `srun Rscript`, and `data.table` thread-control checklist.

Running Stata

Stata on the cluster runs in batch mode. Put temporary files on scratch, match Stata/MP processors to the Slurm CPU request, and close idle sessions because Stata licenses are shared.

```bash
#!/bin/bash
#SBATCH --job-name=stata_run
#SBATCH --partition=default_queue
```

```bash
#SBATCH --time=01:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --output=logs/%x_%j.out

set -euo pipefail

module purge
module load stata

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export MKL_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export STATATMP=/gpfs/scratch60/$USER/stata-tmp/${SLURM_JOB_ID}

mkdir -p "$STATATMP" logs
trap 'rm -rf "$STATATMP"' EXIT

cd /gpfs/project/<proj>/code
stata-mp -b do src/main.do
```

Use a do-file preamble like:

```
capture log close _all
log using "logs/main_${SLURM_JOB_ID}.log", replace text

local ncpus : env SLURM_CPUS_PER_TASK
if "`ncpus'" == "" local ncpus 1
set processors `ncpus'

local statatmp : env STATATMP
di "STATATMP=`statatmp'"

set more off
```

Each Stata array task should write a separate output file. Use `compress`, drop unneeded variables before merges, and keep `tempfile` intermediates out of `$HOME`.

> Claude Code skill: `running-stata`
> `running-stata` covers batch invocation, log handling, `STATATMP`, MP-core sizing, and the license-pool etiquette.

GPUs

Request a GPU only when your code actively uses CUDA — PyTorch, JAX, TensorFlow, RAPIDS, CuPy, or a CUDA kernel you wrote yourself. A GPU held by pure-NumPy, Stata, ordinary dataframe work, downloading, tokenization, or scraping is the most expensive form of resource hoarding on the cluster.

Start with one GPU:

```bash
#!/bin/bash
#SBATCH --job-name=train
#SBATCH --partition=gpunormal
```

```
#SBATCH --gres=gpu:1
#SBATCH --time=01:00:00
#SBATCH --cpus-per-task=4
#SBATCH --mem=32G
#SBATCH --output=logs/%x_%j.out

set -euo pipefail

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export MKL_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export PYTHONUNBUFFERED=1

cd /gpfs/project/<proj>/code
nvidia-smi
nvidia-smi --query-
gpu=timestamp,utilization.gpu,utilization.memory,memory.used,power.draw \
  --format=csv -l 10 > logs/gpu_${SLURM_JOB_ID}.csv &
srun .venv/bin/python train.py
```

Only request multiple GPUs if the code explicitly uses multiple GPUs. Split CPU preprocessing from GPU training: make a CPU job for download, cleaning, tokenization, and feature construction, then submit the GPU job with `--dependency=afterok:<cpu-jobid>`. If `GPU-Util` stays near 0%, VRAM is 0 MB, or utilization alternates between idle and saturated, cancel and debug before burning more GPU-hours.

Current rough guide, to be verified with `sinfo`: `gpunormal` contains RTX 8000/A100-class GPUs; A100 nodes hold three GPUs. RTX 8000/A40 are 48 GB nominal, with RTX 8000 reporting roughly 46 GB usable. The `h100` partition is one node with four 80 GB H100s and is the scarcest resource on the cluster. GPU nodes currently support the CUDA 12.8 runtime, so PyTorch/JAX wheels with bundled CUDA often work without `module load cuda`; load a CUDA module when you need `nvcc` or a specific toolkit:

```
module spider cuda
module load cuda
```

`nvidia-smi` only works inside a GPU allocation, not on the login node. Use `torch.cuda.is_available()` or equivalent framework checks at the start of the training script.

The cardinal sin: idle interactive GPUs

The single worst etiquette violation on the cluster is allocating a GPU interactively and then leaving it open while you go to lunch, to sleep, or to a meeting. Cancel interactive GPU sessions the moment you stop typing — `exit` from inside, or `scancel <job-id>` from outside.

> Claude Code skill: `using-gpus`
> `using-gpus` walks through GPU partition selection, monitoring, CUDA module choices, preprocessing splits, OOM handling, and the idle-GPU detection patterns Claude Code uses to flag wasted allocations.

Working with large data

If your data fits comfortably in your job's RAM, simple tools are fine. Once it stops fitting, query before loading and store reusable data in columnar formats. Parquet is smaller and faster than

CSV, supports column pruning and predicate pushdown, and avoids GPFS metadata storms. DuckDB, Polars lazy scans, and Arrow can stream over datasets larger than the job's RAM.

Inspect files before writing a large script:

```
duckdb -c "DESCRIBE SELECT * FROM 'data.csv'"
duckdb -c "SELECT * FROM 'data.csv' LIMIT 10"
duckdb -c "COPY (SELECT * FROM 'data.csv') TO 'data.parquet' (FORMAT PARQUET)"
```

DuckDB example:

```
import duckdb
con = duckdb.connect()
con.execute("""
COPY (
  SELECT firm_id, year, AVG(ret) AS mean_ret
  FROM read_parquet('/gpfs/project/<proj>/data/returns/*.parquet')
  WHERE year BETWEEN 1990 AND 2020
  GROUP BY firm_id, year
) TO '/gpfs/project/<proj>/output/mean_ret.parquet' (FORMAT PARQUET)
""")
```

Polars lazy example:

```
import polars as pl

result = (
    pl.scan_parquet("/gpfs/project/<proj>/data/panel/*.parquet")
    .filter(pl.col("fyear") >= 2010)
    .select(["gvkey", "fyear", "assets", "sales"])
    .group_by("fyear")
    .agg(pl.col("sales").mean())
)
result.sink_parquet("/gpfs/project/<proj>/output/sales_by_year.parquet")
```

R Arrow example:

```
library(arrow)
library(dplyr)

ds <- open_dataset("/gpfs/project/<proj>/data/panel")
result <- ds |>
  filter(fyear >= 2010) |>
  select(gvkey, fyear, assets, sales) |>
  group_by(fyear) |>
  summarise(mean_sales = mean(sales, na.rm = TRUE)) |>
  collect()
write_parquet(result, "/gpfs/project/<proj>/output/sales_by_year.parquet")
```

Use SQLite with WAL for small local caches and lookup tables; use one connection per process and keep writes single-writer:

```
import sqlite3
conn = sqlite3.connect("/gpfs/project/<proj>/cache/lookup.db")
conn.execute("PRAGMA journal_mode=WAL")
conn.execute("CREATE TABLE IF NOT EXISTS cache (key TEXT PRIMARY KEY, value TEXT)")
conn.commit()
```

Always sample first — a 10,000-row sample is enough to debug code, estimate memory, and check column names. For arrays, write one Parquet per task and combine later with `pl.scan_parquet("output/task_*.parquet")` or DuckDB.

> Claude Code skills: `working-with-large-data`, `accelerating-python`
> `working-with-large-data` covers Parquet conversion, DuckDB, Polars, Arrow, SQLite caches, sample-first workflows, and chunked pipelines. `accelerating-python` covers when to add Numba or parallelism *after* you have fixed the data layer.

Acquiring data: WRDS, APIs, scraping

When a job downloads data — from WRDS, a REST API, a paid LLM API, or a public website — three HPC-specific issues show up that do not exist on a laptop: every job shares one outbound IP, downloads are expensive to repeat, and credentials must never land in scripts or Git.

For WRDS or Postgres, keep connection details in `~/.pg_service.conf` and secrets in `~/.pgpass`, both `chmod 600`, then connect by service name:

```
# ~/.pg_service.conf
[wrds]
host=wrds-pgdata.wharton.upenn.edu
port=9737
dbname=wrds
user=yourwrdsid
```

```
import psycopg
with psycopg.connect("service=wrds") as conn, conn.cursor() as cur:
    cur.execute("select permno, date, ret from crsp.msf where date >= %s",
("2010-01-01",))
    rows = cur.fetchall()
```

Do not run the same WRDS extract inside every analysis job. Download once to project storage, store the raw extract, then analyze local Parquet files. When parallel workers share a database, use `psycopg_pool.ConnectionPool`, create pools inside worker processes, and bound `max_size` deliberately; naive parallelism can exceed WRDS or Postgres connection limits.

```
from psycopg_pool import ConnectionPool

# Create inside the worker process if using multiprocessing.
pool = ConnectionPool("service=wrds", min_size=1, max_size=4)

def fetch_permno(permno):
    with pool.connection() as conn, conn.cursor() as cur:
        cur.execute("select date, ret from crsp.msf where permno = %s", (permno,))
        return cur.fetchall()
```

Cache paid API calls, web pages, and slow endpoints by a hash of the request payload, and write the cache atomically:

```
import hashlib, json
from pathlib import Path

CACHE = Path("/gpfs/project/<proj>/cache/api")
CACHE.mkdir(parents=True, exist_ok=True)
```

```python
def cache_key(payload: dict) -> str:
    return hashlib.sha256(json.dumps(payload, sort_keys=True).encode()).hexdigest()

def cached_call(payload: dict):
    path = CACHE / f"{cache_key(payload)}.json"
    if path.exists():
        return json.loads(path.read_text())
    response = call_expensive_api(payload)
    tmp = path.with_suffix(".json.tmp")
    tmp.write_text(json.dumps(response))
    tmp.rename(path)
    return response
```

Respect rate limits and `robots.txt`, add retries with exponential backoff, and add deliberate sleeps when scraping:

```python
import time, requests

session = requests.Session()

def fetch(url, attempts=5):
    for attempt in range(attempts):
        r = session.get(url, timeout=30)
        if r.status_code == 429 and attempt < attempts - 1:
            time.sleep(int(r.headers.get("Retry-After", "0") or "0") or 2 ** attempt)
            continue
        r.raise_for_status()
        time.sleep(1.0)  # deliberate politeness delay for scraping
        return r
```

For paid APIs or LLM calls, enforce a cost cap in code before submitting a long job:

```python
MAX_BUDGET_DOLLARS = 50.0
spent = 0.0
for request in requests_to_make:
    if spent >= MAX_BUDGET_DOLLARS:
        raise RuntimeError(f"budget exceeded: ${spent:.2f}")
    result = cached_call(request)
    spent += estimate_cost(result)
```

Store raw HTML/JSON responses before parsing so a parser change does not force another download.

> Claude Code skill: `acquiring-data`
> `acquiring-data` covers WRDS service files, connection pooling, API caches, rate limits, retries, cost caps, scraping etiquette, and the credentials pattern in detail.

Starting a new project

A reproducible project layout under `/gpfs/project/<proj>/` saves hours later. The skeleton we recommend is:

```
/gpfs/project/myproj/
├── code/                    # Git repo, pushed to GitHub
│   ├── README.md
```

```
│   ├── pyproject.toml
│   ├── uv.lock
│   ├── renv.lock
│   ├── src/
│   ├── scripts/
│   └── slurm/
├── data/
│   ├── raw/                # original inputs, read-only after ingest
│   └── derived/            # rebuildable intermediates
├── output/                 # results, regeneratable from code+data
├── logs/                   # Slurm logs (%x_%j.out)
├── cache/                  # request-hash caches
└── environments/           # project-local env state not committed
```

Git tracks `code/` and lockfiles, not data, outputs, logs, caches, or environments. A minimal `.gitignore` excludes `.venv/`, `renv/library/`, `data/`, `output/`, `logs/`, `cache/`, `.env`, `*.out`, and `*.err`. The README should state what the project does, where raw data comes from, how to rebuild outputs, and which Slurm script is the first test job. A thin `Justfile` or `Makefile` with commands such as `setup`, `test`, `submit-test`, and `clean-scratch` is often enough.

After ingest, make raw data read-only by convention or permission; any modified data should be written to `data/derived/`. Outputs should be regeneratable — deleting `output/` and re-running should rebuild it. Use one Python/R environment per project rather than a shared `$HOME` environment.

> Claude Code skill: `starting-a-new-project`
> Ask Claude Code to "set up a new project under `/gpfs/project/`" and it will create this layout, initialize Git and uv, write ignores and a README skeleton, and add a starter sbatch script.

HPC etiquette

The HPC is shared and `squeue` is public. If you are a poor citizen of the cluster, expect emails from colleagues waiting on the resources you are holding.

The most expensive form of poor citizenship is letting resources sit idle — an interactive session left open overnight, a batch job whose 24-hour walltime overshoots a 30-minute task, or an interactive GPU allocated and then abandoned for a meeting. Cancel idle sessions immediately with `scancel <job-id>`; an over-allocation that finishes early can be cancelled the same way without waiting for the wallclock to catch up.

Three other patterns matter enough to call out. Do not hog: requesting 64 CPUs and 256 GB when the job needs 4 and 16 keeps your job in the queue longer *and* blocks others. Do not trash GPFS metadata: workflows that write millions of tiny files slow `ls` for every user on the cluster, including the sysadmins, and the right counter-pattern is Parquet plus tarballs (Section 7). And do not get the cluster IP-blocked — every job leaves the network from the same outbound IP, so an aggressive scraper or API client breaks downloads for everyone.

A practical sub-rule for collaborative work: email `somit@yale.edu` for a shared `/gpfs/project/<name>/` folder rather than scattering files across personal directories. Shared folders carry proper permissions, reduce duplication, and survive when one team member's account is offboarded.

Working with Claude Code on the HPC

This guide has a companion: the Yale SOM HPC marketplace (`yale-som-hpc/claude-code-marketplace`), a Claude Code plugin that ships a set of skills — instructions written for the AI — mirroring the sections of this document. When you ask Claude Code to do something on the cluster, it loads the matching skill and follows the conventions encoded there.

The two artifacts are deliberately complementary. This document teaches you, the human, the cluster's mental model and the operational rules well enough to read Claude's plans critically and to operate without it. The marketplace teaches the AI you delegate to, so that when you say "submit this as a Slurm job," Claude already knows about thread exports, partition selection, atomic writes, the cardinal sin of idle GPUs, and the rest. Neither replaces the other: the human who does not understand the cluster cannot evaluate the AI's output, and the AI without the marketplace will produce plausible-looking instructions that violate cluster conventions.

The map from "I want to…" to which skill loads is given in Table 3.

"I want to…"	*Skill*
Get oriented on the cluster	`overview`
Set up SSH keys, Jupyter tunnels	`connecting-securely`
Submit, array, or chain Slurm jobs	`managing-jobs`
Check whether my last job was wasteful	`self-diagnosing-resource-use`
Choose where to store files on GPFS	`using-the-filesystem`
Install software / fix a GLIBC error	`installing-software`
Create a new project layout	`starting-a-new-project`
Run a Python job	`running-python`
Parallelize Python work	`parallel-python`
Speed up slow Python	`accelerating-python`
Run an R job	`running-r`
Run a Stata job	`running-stata`
Request a GPU	`using-gpus`
Process larger-than-memory data	`working-with-large-data`
Download data, call APIs, query WRDS	`acquiring-data`

Table 3: Map from research task to the skill Claude Code will load.

For current install instructions, see the marketplace `README.md`. The short version, from inside Claude Code:

```
/plugin marketplace add yale-som-hpc/claude-code-marketplace
/plugin install hpc@yale-som-hpc
```

Getting help

The commands and contacts you will reach for most often are short enough to keep in one place. For job state and history, use `squeue --me` for what is running now, `squeue --me --start`

and `scontrol show job <id>` for diagnosing why something is pending, and `seff <id>` together with `sacct -j <id>` for resource usage after a run. For partition and node detail, `sinfo -s` is the live view. For account, storage, and project-folder requests, email `somit@yale.edu`. Reference documentation lives at slurm.schedmd.com for Slurm and lmod.readthedocs.io for the module system, and the marketplace skills serve as a structured second source for the operational patterns this guide describes.

Appendix: example files

The files below form a complete first job — a Python script, a Slurm submission script, and the commands to submit and inspect the run. They are designed to be copied verbatim, run end-to-end, and then modified.

fibonacci.py

```python
#!/usr/bin/env python3
"""
Compute Fibonacci numbers. A trivial Python script for testing
HPC submission end-to-end.
"""

import sys
import time

def fibonacci(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

def fibonacci_sequence(n):
    return [fibonacci(i) for i in range(n)]

if __name__ == "__main__":
    n = int(sys.argv[1]) if len(sys.argv) > 1 else 20
    print(f"Computing first {n} Fibonacci numbers...")
    t0 = time.time()
    seq = fibonacci_sequence(n)
    elapsed = time.time() - t0
    for i, fib in enumerate(seq):
        print(f"F({i}) = {fib}")
    print(f"\nDone in {elapsed:.4f} s. F({n}) = {fibonacci(n)}")
```

submit_fibonacci.sh

```bash
#!/bin/bash
#SBATCH --job-name=fibonacci
#SBATCH --partition=default_queue
#SBATCH --output=logs/fibonacci_%j.out
#SBATCH --error=logs/fibonacci_%j.err
#SBATCH --time=00:10:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G

set -euo pipefail

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export MKL_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export OPENBLAS_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export NUMEXPR_NUM_THREADS=${SLURM_CPUS_PER_TASK:-1}
export PYTHONUNBUFFERED=1
```

```bash
# Print job information
echo "Job started at: $(date)"
echo "Running on node: $(hostname)"
echo "Job ID: $SLURM_JOB_ID"
echo "========================================="

# Load Python and run the script.
module load python
srun python fibonacci.py 30

echo "========================================="
echo "Job finished at: $(date)"
```

Running it

```bash
mkdir -p logs
sbatch submit_fibonacci.sh
squeue --me
# when the job finishes:
seff <job-id>
cat logs/fibonacci_<job-id>.out
```

The `seff` call at the end is the habit worth building from day one.